**МИНОБРНАУКИ РОССИИ**

федеральное государственное бюджетное образовательное учреждение высшего образования

**«Национальный исследовательский университет «МЭИ»**

---

**Задание на лабораторную работу №3**

**по дисциплине «Вычислительные системы»**

# «Работа кэш-памяти в вычислительной системе»

## Цель

Целью лабораторной работы является исследование влияния иерархии памяти на характеристики выполнения программ посредством разработки и реализации модели кэш-памяти, интегрированной в ранее созданную вычислительную систему, а также количественная оценка эффективности различных параметров и политик организации кэша.

## Теория

В вычислительных системах производительность выполнения программ определяется не только скоростью арифметических операций процессора, но и временем доступа к памяти. Разница между латентностью операций центрального процессора и латентностью доступа к основной памяти может составлять несколько порядков величины. Данное явление принято обозначать как «разрыв производительности процессора и памяти» (часто называемое «memory wall»). Для уменьшения средней задержки доступа используется иерархическая организация памяти, при которой между процессором и основной памятью располагаются промежуточные уровни хранения данных меньшего объёма, но существенно более высокой скорости доступа. Эти уровни называются кэш-памятью.

Кэш-память представляет собой аппаратно или программно реализованный буфер, содержащий копии блоков данных, находящихся в более медленном уровне памяти. Основная идея кэширования состоит в том, что обращения программы к памяти обладают статистическими свойствами локальности, благодаря которым небольшая часть адресного пространства используется значительно чаще, чем остальная. Использование кэша позволяет обслуживать значительную долю обращений к памяти на более быстром уровне и тем самым уменьшать среднюю латентность доступа.

С точки зрения формальной модели памяти адресное пространство основной памяти можно рассматривать как последовательность адресуемых ячеек. Кэш-память хранит копии фрагментов этого пространства фиксированного размера, называемых блоками или строками кэша. Пусть размер блока равен B байт. Тогда вся память разбивается на последовательность блоков, каждому из которых соответствует диапазон адресов основной памяти.

Каждая строка кэша содержит не только данные блока, но и служебную информацию, позволяющую определить соответствие между содержимым строки и блоком основной памяти. К такой информации относятся как минимум:

- признак валидности (valid bit), показывающий, содержит ли строка корректные данные;
- тег (tag), представляющий старшие биты адреса блока основной памяти;
- при использовании политики write-back — признак модификации (dirty bit), показывающий, были ли данные строки изменены относительно основной памяти.

При обращении процессора к памяти происходит отображение адреса на структуру кэша. Для формализации данного процесса адрес обычно представляется как конкатенация трёх полей:

- смещения внутри строки (offset)
- индекса набора (index)
- тега (tag)

Смещение определяет положение данных внутри строки, индекс используется для выбора набора строк кэша, а тег позволяет проверить, соответствует ли найденная строка требуемому блоку основной памяти.

В зависимости от способа отображения блоков основной памяти на строки кэша различают несколько схем организации кэша.

В кэше с прямым отображением (direct-mapped cache) каждому блоку основной памяти соответствует ровно одна строка кэша. Индекс строки определяется частью адреса блока. Такая организация обладает простой логикой доступа, однако может приводить к конфликтным промахам, когда несколько активно используемых блоков отображаются в одну и ту же строку.

В множественно-ассоциативной организации (set-associative cache) строки кэша группируются в *наборы*. Каждый блок основной памяти может размещаться в любой строке соответствующего набора. Если в наборе содержится k строк, такая организация называется k-кратно ассоциативной. Данный подход уменьшает вероятность конфликтных промахов за счёт увеличения гибкости размещения блоков.

В полностью ассоциативном кэше (fully associative cache) любой блок основной памяти может быть размещён в любой строке кэша. Такая организация минимизирует конфликтные промахи, однако требует более сложных механизмов поиска строки по тегу.

При заполнении кэша возникает необходимость выбора строки, которая будет вытеснена для размещения нового блока. Для этого используются политики замещения. Наиболее распространёнными являются:

- FIFO (First In First Out) - вытесняется строка, находящаяся в кэше наибольшее время;
- LRU (Least Recently Used) - вытесняется строка, к которой дольше всего не происходило обращений;
- Случайная - вытесняется случайно выбранная строка.

Выбор политики замещения влияет на вероятность возникновения промахов и, следовательно, на общую эффективность кэш-подсистемы.

Кроме политики замещения важную роль играет политика записи данных. В режиме write-through каждая операция записи одновременно обновляет как кэш, так и основную память. В режиме write-back изменения сначала фиксируются только в кэше, а запись в основную память выполняется при вытеснении строки. Write-back уменьшает число обращений к основной памяти, однако требует отслеживания модифицированных строк посредством dirty-битов.

Для анализа эффективности кэш-памяти используются количественные характеристики. Пусть N - общее число обращений к памяти, H - число попаданий в кэш, а M - число промахов. Тогда коэффициент промахов определяется как

$$MissRate = M / N$$

а коэффициент попаданий равен

$$HitRate = H / N$$

Среднее время доступа к памяти (Average Memory Access Time) определяется выражением

$$AMAT = HitTime + MissRate \times MissPenalty$$

где HitTime — время доступа при попадании, а MissPenalty — дополнительная задержка при промахе, связанная с извлечением блока из более медленного уровня памяти.

Промахи кэша принято классифицировать по причинам их возникновения. Обязательные (compulsory) промахи возникают при первом обращении к блоку памяти. Конфликтные (conflict) промахи связаны с ограничениями схемы отображения блоков в кэш. Ёмкостные (capacity) промахи возникают, когда рабочее множество программы превышает объём кэша.

При интеграции кэш-памяти в конвейерную модель процессора операции чтения и записи памяти обычно выполняются на стадии MEM. В предыдущей

лабораторной работе стадия MEM рассматривалась как однотактовая операция доступа к памяти. В данной работе это предположение интерпретируется как модель доступа при попадании в кэш. Таким образом, при попадании операция завершается за один такт стадии MEM, тогда как при промахе возникает дополнительная задержка, моделируемая как останов конвейера на величину штрафа за промах. Если обращение приводит к попаданию в кэш, операция завершается за один такт стадии MEM. В случае промаха происходит обращение к основной памяти, что приводит к задержке выполнения конвейера на величину штрафа за промах. В течение этой задержки новые инструкции не могут продвигаться по конвейеру, что приводит к увеличению общего числа тактов выполнения программы.

Таким образом, модель кэш-памяти позволяет исследовать влияние параметров организации памяти, структуры адресации и характеристик программных обращений на эффективность выполнения программ в вычислительной системе.

**Формальная модель отображения адресов**

Пусть объём кэша равен C байт, размер строки кэша равен B байт, а ассоциативность равна A. Тогда число строк кэша равно C / B, а число наборов равно

$$S = C / (B \times A)$$

Пусть адрес в основной памяти имеет длину m бит. Для обращения к кэшу адрес разбивается на три поля: offset, index и tag

Число бит смещения определяется размером строки:

$$offset_bits = log2(B)$$

Число бит индекса определяется числом наборов:

$$index_bits = log2(S).$$

Оставшиеся старшие биты адреса образуют тег:

$$tag_bits = m - offset_bits - index_bits.$$

1. При обращении к памяти выполняется следующий алгоритм:
2. По полю index выбирается набор строк кэша.
3. В выбранном наборе выполняется сравнение тегов всех строк с полем tag.
4. Если найдена строка с совпадающим тегом и установленным признаком valid, фиксируется попадание.

В противном случае фиксируется промах, после чего соответствующий блок основной памяти загружается в одну из строк набора согласно политике замещения.

**Влияние параметров кэша на эффективность**

Эффективность кэш-подсистемы определяется как характеристиками самой памяти, так и свойствами программы. Основными параметрами организации кэша, влияющими на вероятность промахов, являются размер кэша, размер строки и степень ассоциативности.

Увеличение объёма кэша, как правило, приводит к уменьшению ёмкостных промахов, поскольку большее рабочее множество программы может быть размещено в быстром уровне памяти. Однако рост объёма кэша может увеличивать время доступа и усложнять аппаратную реализацию.

Размер строки кэша влияет на использование пространственной локальности. При увеличении размера строки в кэш загружается больший фрагмент соседних данных, что может уменьшать число промахов при последовательном доступе. Однако слишком большой размер строки может приводить к увеличению штрафа за промах и к неэффективному использованию кэша, если программа обращается только к небольшой части блока.

Ассоциативность кэша определяет гибкость размещения блоков памяти. Увеличение ассоциативности уменьшает вероятность конфликтных промахов,

однако увеличивает сложность поиска строки по тегу и может увеличивать латентность обращения к кэшу.

Таким образом, параметры кэша представляют собой компромисс между аппаратной сложностью, латентностью доступа и вероятностью промахов. Моделирование кэш-памяти позволяет исследовать влияние этих параметров на характеристики выполнения программ и выявить оптимальные конфигурации для различных типов нагрузок.

**Задание**

В рамках лабораторной работы требуется расширить разработанную ранее модель вычислительной системы добавлением уровня кэш-памяти, расположенного между процессором и основной памятью. Реализованная модель должна корректно интегрироваться в существующую модель выполнения инструкций и учитывать влияние кэш-подсистемы на временные характеристики обращения к памяти.

Кэш-память должна моделироваться как отдельная структура данных, содержащая строки фиксированного размера, служебные поля (как минимум valid и tag), а также блок данных основной памяти. Реализация должна обеспечивать алгоритм отображения адресов основной памяти на строки кэша в соответствии с формальной моделью, описанной в теоретической части.

При каждом обращении процессора к памяти должна выполняться проверка наличия соответствующего блока в кэше. В случае попадания операция доступа считается завершённой за время доступа к кэшу. В случае промаха должен выполняться алгоритм загрузки блока из основной памяти с учётом выбранной политики размещения и замещения.

Модель должна учитывать штраф за промах. Если обращение приводит к промаху, выполнение конвейера должно быть приостановлено на заданное число тактов, необходимое для извлечения блока из основной памяти. После завершения загрузки блока выполнение программы продолжается.

**Требования к реализации**

Реализованная модель кэш-памяти должна поддерживать:

- фиксированный размер строки кэша;
- заданный общий объём кэша;
- параметр ассоциативности;
- одну из политик замещения (FIFO, LRU или случайную);
- корректную обработку операций чтения и записи.

Политика записи может быть выбрана разработчиком (write-through или write-back), однако она должна быть явно указана в описании реализации.

Все параметры кэш-памяти должны задаваться конфигурацией модели и не должны быть жёстко зафиксированы в коде.

**Экспериментальная часть**

После реализации модели кэш-памяти необходимо выполнить программу, использовавшуюся в предыдущих лабораторных работах, и провести измерение характеристик выполнения.

Для выполненной программы необходимо определить:

1. общее число обращений к памяти;
2. число попаданий в кэш;
3. число промахов;
4. коэффициент промахов (miss rate);
5. общее число тактов выполнения;
6. значение CPI.

После этого требуется провести серию экспериментов, изменяя параметры кэша (объём кэша, размер строки и ассоциативность) и сравнить полученные характеристики выполнения.

## Контрольные вопросы

1. В чём заключается различие между временной и пространственной локальностью обращений к памяти?
2. Чем отличается кэш с прямым отображением от множественно-ассоциативного кэша?
3. Как разложение адреса на поля tag, index и offset связано со структурой кэша?
4. Какие причины приводят к возникновению compulsory, conflict и capacity промахов?
5. В чём заключается различие между политиками записи write-through и write-back?
6. Как изменение размера строки кэша влияет на вероятность промахов?
7. Почему увеличение ассоциативности уменьшает вероятность конфликтных промахов?
8. Каким образом штраф за промах влияет на среднее время доступа к памяти?
9. Как кэш-память влияет на значение CPI в конвейерной модели выполнения инструкций?
10. Какие параметры кэша оказывают наибольшее влияние на эффективность выполнения программ?